

SECURITY REVIEW

FOR FACTCHECK.FUN



INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	SCOPE	04
03	AUDITORS	05
04	METHODOLOGY	06
05	RISK CLASSIFICATION	07
06	FINDINGS SUMMARY	08
07	DETAILED FINDINGS	09
	M-01 Signature Verification Broken Under EIP-7702 Due to Reliance on isContract Logic	09
	M-02 Lack of Minimum feeRate Check May Allow Orders to be fulfilled without paying any fees	10
	I-01 Missing Market ID Consistency Check in settleMatchedOrders	11
	I-02 Ensure Taker and Maker Orders Are on Opposite Sides	12
	I-03 Old Treasury Doesn't Get Revoked When Updated To New One	13
08	DISCLAIMER	14
09	ABOUT KANN AUDITS	15

AUDIT SUMMARY

factcheck.fun engaged Kann Audits to perform a security review of the factcheck.fun smart contracts. A team of 3 security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

5
ISSUES

● CRITICAL	-
● HIGH	-
● MEDIUM	02
● LOW	-
● INFORMATIONAL	03

ENGAGEMENT DETAILS

PREPARED FOR factcheck.fun

PREPARED BY Kann Audits

DATE REPORTED 28 May 2025

METHODS Manual review

02 - WHAT WE REVIEWED

SCOPE

#	FILE
01	/src/FactCheckExchange.sol

03 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

KA

Kann

LEAD SECURITY RESEARCHER

IV

Ivan Fitro

LEAD SECURITY RESEARCHER

PE

Pelz

ASSOCIATE SECURITY RESEARCHER

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

06 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL -	● HIGH -	● MEDIUM 02	● LOW -	● INFO 03
TOTAL 05 FIXED 02 ACKNOWLEDGED 03				

ID	TITLE	SEVERITY	STATUS
M-01	Signature Verification Broken Under EIP-7702 Due to Reliance on isContract Logic	● MEDIUM	● ACKNOWLEDGED
M-02	Lack of Minimum feeRate Check May Allow Orders to be fulfilled without paying any fees	● MEDIUM	● FIXED
I-01	Missing Market ID Consistency Check in settleMatchedOrders	● INFORMATIONAL	● ACKNOWLEDGED
I-02	Ensure Taker and Maker Orders Are on Opposite Sides	● INFORMATIONAL	● ACKNOWLEDGED
I-03	Old Treasury Doesn't Get Revoked When Updated To New One	● INFORMATIONAL	● FIXED

DETAILED FINDINGS

M-01

Signature Verification Broken Under EIP-7702 Due to Reliance on isContract Logic

SEVERITY

● MEDIUM

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

In the FactCheckExchange.sol contract, the settleMatchedOrders() function attempts to validate orders using signatures by distinguishing EOAs from contracts. It uses the following logic:

-If the address has code (isContract returns true), it calls isValidSignature() via ERC-1271.

-If the address has no code, it assumes it's an EOA and uses ECDSA.recover() to verify the signature.

This approach breaks under the upcoming Ethereum Pectra upgrade, which includes EIP-7702. EIP-7702 allows EOAs to temporarily attach code during a transaction, meaning any externally owned account can now appear as a contract at runtime.

As a result:

A 7702-enabled EOA with temporary code will cause the logic to treat it as a contract.

If the code does not implement ERC-1271, the isValidSignature() call will fail.

The logic does not fall back to ECDSA.recover(), causing legitimate EOA signatures to be rejected.

■ REMEDIATION

● ACKNOWLEDGED

M-02

Lack of Minimum feeRate Check May Allow Orders to be fulfilled without paying any fees

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

The `settleMatchedOrders` function allows the backend to settle matched orders that were created and matched off-chain. While the contract enforces an upper bound for `feeRate`, it does not enforce a minimum fee rate.

This omission allows orders with extremely low `feeRate` values (e.g., 1 wei) to be matched and settled. As a result, users can effectively bypass protocol fees altogether, undermining protocol revenue and fairness.

Since order matching is handled off-chain and executed on-chain by an address with the `BACKEND_ROLE`, malicious or careless matching could lead to widespread abuse of low-fee orders.

■ RECOMMENDATION

Introduce a `minFeeRate` variable, configurable by protocol administrators, and enforce it during order validation to ensure all orders pay at least a minimum protocol fee:

```
require(order.feeRate ≥ minFeeRate, "Fee rate below minimum allowed");
```

■ REMEDIATION

● FIXED

I-01

Missing Market ID Consistency Check in settleMatchedOrders

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

The settleMatchedOrders function doesn't enforce that all makerOrders share the same marketId as the provided takerOrder. While it correctly verifies signature validity and array length consistency, it does not validate that each makerOrder.marketId equals takerOrder.marketId.

■ RECOMMENDATION

```
for (uint i = 0; i < makerOrders.length; i++) {
    require(
        makerOrders[i].marketId == takerOrder.marketId,
        "FactCheckExchange: Market ID mismatch"
    );
}
```

■ REMEDIATION

● ACKNOWLEDGED

I-02

Ensure Taker and Maker Orders Are on Opposite Sides

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

In the `settleMatchedOrders` function, there is currently no check to enforce that the `takerOrder` and each `makerOrder` are on opposite sides of the trade (i.e., one is a buyer, the other is a seller). While it may be assumed that the backend prevents invalid matches, relying solely on off-chain guarantees can introduce hidden risks.

■ RECOMMENDATION

```
require(takerOrder.side !== makerOrder.side, "Orders must be on opposite sides");
```

■ REMEDIATION

● ACKNOWLEDGED

I-03

Old Treasury Doesn't Get Revoked When Updated To New One

SEVERITY

● INFORMATIONAL

STATUS

● FIXED

■ DESCRIPTION

In the `updateTreasuryWallet()` function, the contract sets a new `treasuryWallet` address and grants it the `TREASURY_ROLE`

However, the role from the previous treasury address is not revoked. As a result, any previously assigned treasury address retains the `TREASURY_ROLE`

■ RECOMMENDATION

Revoke the old `treasuryWallet`.

■ REMEDIATION

● FIXED

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits